ACI · AIMLCZG557

Search: Uninformed & Informed (A*, GBFS)

Advanced Computing for AI — Lesson 2

Exam: 28 Jun 2026 · Closed book · A* trace is a guaranteed exam question

How to use this companion. Blue = the idea. Amber = everyday analogy. Green = fully-solved example. Red = common mistake. Purple = one line to remember.

1 Search Problem Formulation

A search problem is the formal way we describe "get from here to there". We define it with six parts: the **state space** (all possible situations), the **initial state** (where we start), **actions** (what we can do), the **transition model** (what results from an action), a **goal test** (is this the destination?), and **path cost** (how expensive is a path?).

☞ The intuition

Think of Google Maps. The state space is every intersection in the city. The initial state is your driveway. Actions are "turn left", "go straight". The transition model says which intersection you reach. The goal test checks whether you arrived at the restaurant. Path cost is total driving time.

The state space is typically implicit — we do not list every state upfront. Instead, we generate states on demand by applying actions. A **node** in the search tree stores: the state, the parent node, the action that created it, and the path cost $g(n)$.

Where this shows up in ML. Neural Architecture Search, hyperparameter optimisation, and planning in model-based RL all reduce to search problems. The state space is the space of architectures or hyperparameter combinations.

✔ Key takeaway

State, Initial state, Actions, Transition model, Goal test, Path cost. A solution is any path to a goal; an optimal solution minimises path cost.

2 Uninformed Search

Uninformed ("blind") strategies have no knowledge about how far the goal is. They differ only in the order they expand nodes.

☞ The intuition

Imagine looking for your keys in a dark house. BFS checks every room on the ground floor before going upstairs (broad first). DFS dives into one room and searches every drawer before moving on (deep first). UCS picks whichever room costs least to reach next.

Breadth-First Search (BFS). Expands shallowest node first. Uses a FIFO queue. Complete (yes, if branching factor is finite). Optimal (yes, for uniform step costs). Time and Space: $O(b^d)$ where b = branching factor, d = depth of shallowest goal.

Depth-First Search (DFS). Expands deepest node first. Uses a LIFO stack. Not complete (can loop). Not optimal. Time: $O(b^m)$, Space: $O(bm)$ — linear, much better than BFS.

Uniform-Cost Search (UCS). Expands the node with lowest path cost $g(n)$. Uses a min-priority queue on g. Complete and optimal. Reduces to BFS when all step costs are equal.

Algorithm	Complete?	Optimal?	Time	Space
BFS	Yes	Yes (uniform)	$O(b^d)$	$O(b^d)$
DFS	No	No	$O(b^m)$	$O(bm)$
UCS	Yes	Yes	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	same
IDS	Yes	Yes (uniform)	$O(b^d)$	$O(bd)$

✗ Watch out

DFS is not complete in infinite or cyclic state spaces — it can loop forever. Always add a visited/closed list when using DFS on graphs. IDS gets the best of both worlds: BFS-like completeness and optimality with DFS-like memory.

✔ Key takeaway

BFS = shallowest first (FIFO); DFS = deepest first (LIFO); UCS = cheapest first (priority queue on g). IDS = best of both: $O(bd)$ memory, $O(b^d)$ time.

3 Heuristics

A **heuristic** $h(n)$ is an estimate of the cost from node n to the goal. It uses domain knowledge to guide search towards the goal.

☞ The intuition

A heuristic is a shortcut born from experience. When navigating a city, "straight-line distance to the destination" is a heuristic — it's not exact (roads aren't straight), but it's a good guess that steers you in the right direction.

Admissibility. A heuristic is **admissible** if it never overestimates the true cost:

$$h(n) \leq h^*(n) \quad \text{for all nodes } n$$

where $h^*(n)$ is the true cost from n to the goal. An admissible heuristic is optimistic — it always thinks the goal is at least as close as it really is.

Consistency (Monotonicity). A heuristic is **consistent** if for every node n and every successor n' reached by action a:

$$h(n) \leq c(n, a, n') + h(n')$$

This is the triangle inequality applied to heuristics. Consistency ⇒ admissibility (but not vice versa).

★ Everyday picture

Straight-line distance (SLD) from a city to the goal is both admissible (roads are never shorter than straight lines) and consistent (going via an intermediate city can only add cost, never subtract it).

Designing admissible heuristics. A reliable method: **solve a relaxed problem** (remove constraints). For 8-puzzle:

- h_1 = number of misplaced tiles (relaxed: tiles can teleport)
- h_2 = sum of Manhattan distances (relaxed: tiles can slide through each other)

Both are admissible. $h_2 \geq h_1$ always, so h_2 *dominates* h_1 and leads to less search.

✔ Key takeaway

Admissible = never overestimates. Consistent = triangle inequality holds. Consistent ⇒ admissible. Relaxed-problem heuristics are always admissible.

4 Greedy Best-First Search (GBFS)

GBFS expands the node that *appears closest* to the goal according to $h(n)$ alone.

$$f(n) = h(n)$$

It uses a priority queue ordered by h. It is fast but not optimal and not complete (can get stuck in loops). It does no "path cost accounting" at all — it just dives greedily toward the goal.

✔ Key takeaway

GBFS: expand node with smallest $h(n)$. Fast but not optimal. Use when you need speed and can tolerate a suboptimal solution.

5 A* Search

A* combines the path cost so far (g) with the heuristic estimate to the goal (h):

$$f(n) = g(n) + h(n)$$

$g(n)$ = cost from start to n (exact). $h(n)$ = estimated cost from n to goal (heuristic). $f(n)$ = estimated total cost of the cheapest path through n.

☞ The intuition

g says "how far have I already travelled?" and h says "how far do I still need to go?". Adding them gives a complete picture. A* balances these two perfectly, so it always finds the optimal path when h is admissible.

Optimality. A* with an admissible heuristic is optimal. Proof sketch: A* will not expand a goal node G' with a higher cost than the optimal goal G, because $f(G') > f(G)$ and A* expands in order of f.

✎ Worked example: Full A* trace on a 6-node graph

Graph edges (undirected, step costs shown):

S–A: 1	A–B: 3
S–D: 4	A–C: 2
D–B: 1	B–G: 2
	C–G: 4

Heuristic values: $h(S) = 6,\ h(A) = 5,\ h(D) = 3,\ h(B) = 2,\ h(C) = 4,\ h(G) = 0.$

Step 1. Initialise. Open = {S: $f = 0 + 6 = 6$}. Closed = {}.

Step 2. Expand S ($f = 6$). Generate A ($g = 1, f = 1 + 5 = 6$) and D ($g = 4, f = 4 + 3 = 7$). Open = {A:6, D:7}. Closed = {S}.

Step 3. Expand A ($f = 6$). Generate B ($g = 4, f = 4 + 2 = 6$) and C ($g = 3, f = 3 + 4 = 7$). Open = {B:6, C:7, D:7}. Closed = {S, A}.

Step 4. Expand B ($f = 6$). Generate G ($g = 6, f = 6 + 0 = 6$). Also D can reach B with $g = 5 > 4$, skip. Open = {G:6, C:7, D:7}. Closed = {S, A, B}.

Step 5. Expand G ($f = 6$). Goal test passes. **Solution found!**

Optimal path: S → A → B → G, total cost = **6**.
Sense-check: alternative S→D→B→G costs $4 + 1 + 2 = 7 > 6$. Correct.

✗ Watch out

A* expands nodes in order of $f = g + h$, not just h. A common exam mistake is to sort by h only (that is GBFS, not A*). Always compute and state g, h, and f for every node. Also: when a node is re-encountered with a lower g, update it — otherwise you may miss the optimal path.

Where this shows up in ML. A* (and its variants ARA*, D*) is the backbone of robot path planning. It appears in NLP as the decoder search in beam search and sequence-to-sequence models. The heuristic in NLP is often a language model score.

✔ Key takeaway

A*: $f(n) = g(n) + h(n)$. Optimal when h is admissible. Always show g, h, f at every step. This is the #1 exam algorithm for ACI.